Ramas ERP

Debugging Guide — how to find a problem and what to change

This guide is written so that anyone can use it: you do not need to be a programmer to find **where** a problem is. A programmer (or Claude) then uses the same pages to fix it.

How to use this guide (3 steps)

1. See — what is wrong on the screen? Find it in the table "What do you see?" (chapter 3).

2. Look — press **F12** in the browser and read the error number (chapter 4 and 5).

3. Open — the module map (chapter 7) tells you which file to open and change.

Project: D:\rr\ramas-modern • 125 screens built • 40 modules • 333 API addresses • 175 automatic tests • made 02/10/2026

Contents

1. The app in one picture
2. Five-minute check: is everything running?
3. What do you see? → where is the problem
4. Using F12 (the browser's "doctor")
5. Error numbers and messages — what they mean
6. Reading the API log
7. Module map — which screen uses which files
8. What to change for what
9. Checking the data in the database
10. Common problems and their fixes
11. How to ask Claude (or anyone) for help
12. Command cheat sheet
13. Where everything lives (folders)
14. Words used in this guide

1. The app in one picture

Ramas has **three parts**. Every problem lives in one of them. Finding the part is half of the fix.

****	**SCREEN (what you see)**	**API (the middle)**	**DATABASE (where data is kept)**
What is it	Pages in the browser: forms, tables, buttons, the dashboard	A small server that checks every request and runs the business rules	SQL Server — tables with dockets, bills, users …
Made with	React + Ant Design (JavaScript)	Node.js + Express (JavaScript)	Microsoft SQL Server (T-SQL)
Lives in	`client/src/pages/…`	`server/src/modules/…`	dev database `ramas_modern_dev`; scripts in `server/db/dev/`
Address on this PC	http://localhost:5173	http://localhost:4000/api/…	`.\SQLEXPRESS`
When it breaks you see	white page, wrong layout, button does nothing	red message, error number (400, 422, 500 …)	error 500 with an SQL message, or wrong numbers

What happens when you click "Save"

1. The **screen** collects what you typed and sends it to the API, e.g. `POST /api/dockets`.
2. The **API** checks it (missing box? wrong date?). If something is wrong it answers with an error number and a message — the red popup you see.
3. If all is fine the API runs SQL on the **database** (insert / update), usually inside one "transaction" (all or nothing).
4. The API answers "OK" (number 200 / 201) and the screen shows the success message.

> **Remember**
>
> Every screen talks to the API through an address that starts with `/api/` — that address tells you which module (folder) to open.
>
> Example: `/api/client-bills/…` → folder `server/src/modules/clientbill/`.

2. Five-minute check: is everything running?

Before looking for a "bug", check these four things. Most "the app is not working" problems are one of them.

Check	How	If it is not OK
1. SQL Server is running	Start menu → "Services" → **SQL Server (SQLEXPRESS)** shows "Running". Or PowerShell: `Get-Service MSSQL$SQLEXPRESS`	Right-click → Start. If it stops again: the PC is out of memory — close programs.
2. The API is running	Open http://localhost:4000/api/health — it must show `{"ok":true}`	Start it: `npm --prefix D:\rr\ramas-modern\server start` (see chapter 12)
3. The screen is running	Open http://localhost:5173 — the login page appears	Start it: `npm --prefix D:\rr\ramas-modern\client run dev`
4. You can log in	Choose login type, **division** and **financial year**, user and password	Wrong year = you see no data; see "Invalid user id or password" in chapter 10

Memory (RAM) matters on this PC

When free RAM is below ~1 GB, SQL Server becomes slow: pages keep loading, "Query timeout", "insufficient system memory (701)".

Fix: close WhatsApp, Postman, extra browser tabs and old Claude sessions, then try again. Check: Task Manager → Performance → Memory.

3. What do you see? → where is the problem

Find the line that matches what you see. The middle column tells you the **part**; the right column tells you **where to look**.

What you see	Part	Where to look / what to do
The page is **white / blank**	SCREEN	F12 → **Console**: a red line names the file and line, e.g. `ClientBillPage.jsx:120`. Open that file in `client/src/pages/`.
A **red message** pops up, e.g. "Select the client" or "Bill date must be in the login financial year"	API (a rule)	Not a crash — the API refused the input. Read the message and correct the entry. To change the rule: search the message text in the module's `*.routes.js` (box checks) or `*.service.js` (business rules).
"**Invalid user id or password**"	API / DATABASE	User, password, division or year wrong. Users are in tables `users`, `ClientLogin_Users`, `Franchisee_Users`.
Page keeps **loading** / "Network Error"	API not running or DATABASE slow	Chapter 2: check http://localhost:4000/api/health. Restart the API. Check free RAM.
A **number looks wrong** (total, count, dashboard chart)	API (SQL in the service)	F12 → Network → the `/api/…` call → Response shows the number the API sent. If wrong there → the `*.service.js` of that module (its SQL). If right there but shown wrong → the page file.
It **saved, but the data in the database is wrong**	API / DATABASE	The insert in `*.service.js`, or a stored procedure in `server/db/dev/NN-*.sql`. Check the row with SQL (chapter 9).
Error **500** "Internal server error"	API or DATABASE	Look at the API window/log: a line `"Unhandled error"` shows the real message (chapter 6).
A **menu item is missing**	DATABASE (menu rights) / SCREEN	Menu rows in `SubMenu` + `Employee_Permission` (`server/db/dev/99-seed.sql`); the page link in `client/src/routes/legacyRoutes.js` and `client/src/App.jsx`.
"**Page not found**"	SCREEN	The address is not in `client/src/App.jsx` (Route list).
A **print page** is empty	SCREEN / API	Print pages are in the same page file (e.g. `OctroiBillPrintRoute`); F12 → Network shows if the bill was not found (404).
A **docket's journey** looks wrong (wrong branch, short packets, late, entered by …)	DATA entry	Open **Tracking** (`/tracking?docket=…`). The **Problems** box and the red cells in **Entries grid** show which entry is wrong and who entered it.
Dashboard shows **no data**	API / DATABASE	Wrong period or "Only this branch" selected; or the database is empty (demo data: chapter 10).

4. Using F12 (the browser's "doctor")

Chrome and Edge have a built-in tool that shows **exactly** what went wrong. It does not change anything — it only shows.

A. When a page is blank or a button does nothing → Console

1. Press **F12** (or right-click → Inspect).
2. Click the **Console** tab.
3. Look for **red** lines. They name a file and line number, e.g. `DashboardPage.jsx:212`. Take a screenshot.

B. When you get an error message or wrong data → Network

1. Press **F12** → click **Network**.
2. Do the action again (click Save / Search).
3. In the filter box type **api**. Rows in **red** failed.
4. Click the row. **Headers** shows the address (e.g. `/api/challans`) and the **Status** number (e.g. 422).
5. **Payload** shows what the screen sent. **Response** shows what the API answered — the error message is here.

The address tells you the folder

`/api/dockets/…` → `server/src/modules/docket/` `/api/client-bills/…` → `server/src/modules/clientbill/`

`/api/dashboard/…` → `server/src/modules/dashboard/` `/api/vendor-invoices/…` → `server/src/modules/vendorinvoice/`

The full list is the module map in chapter 7.

5. Error numbers and messages — what they mean

Every answer from the API has a number. **200 / 201 = OK.** Anything from 400 up is a problem:

Number	Meaning in plain words	What to do
400	Something you typed is missing or in the wrong form (e.g. no date, letters in a number box).	Read the message and fix the entry. The checks are in the module's `*.routes.js` (zod).
401	Not logged in, or the login expired (after ~10 hours).	Log in again.
403	Logged in, but this user may not do this (e.g. a client opening a staff page).	Use the right user; rights are per role / menu.
404	Not found: wrong number, or it belongs to another branch / financial year.	Check the number, the branch selected at the top, and the year you logged in with.
409	Conflict: it is already done / already used / cancelled (e.g. "already billed").	Normal protection. Undo the earlier step first if it was a mistake.
422	A business rule stopped it (e.g. "Ledgers missing", "advance more than freight").	Read the message; often a master is missing (ledger, rate, user right).
429	Too many login tries (20 in 15 minutes).	Wait 15 minutes.
500	The API crashed on this request — a real bug or a database problem.	Read the API log line "Unhandled error" (chapter 6).
502 / 504 or "Network Error"	The API is not running or did not answer.	Chapter 2: start the API; check memory.

Database messages inside a 500 error

Message contains	Meaning	Fix
`Query timeout expired`, `insufficient system memory`, `701`	SQL Server is out of memory / too slow	Free RAM (close programs), try again. Do not change SQL Server memory settings.
`Invalid column name` / `Invalid object name`	The database is missing a column/table the code needs	Apply the missing `server/db/dev/NN-*.sql` file (chapter 12).
`Cannot insert duplicate key`	That number/code already exists	Use another number, or find why it was taken twice.
`String or binary data would be truncated`	A value is longer than the column allows	Shorten the value or widen the column in a new dev SQL file.
`Login failed for user`	The API cannot log in to SQL Server	Check `server/.env` (`DB_*` settings).
`operation timed out for an unknown reason`	Waiting for a free database connection	The database is slow (memory) — free RAM and retry.

6. Reading the API log

The window where the API runs (or `D:\rr\tunnel\server-error.log` for the test link) prints one line per request:

```
POST /api/challans 422 15.8 ms - 67
GET /api/dashboard/overview?from=2026-07-04&to=2026-10-01 200 412.3 ms - 12677
```

- **POST / GET** = what kind of request, then the **address**, then the **number** (chapter 5), then how long it took.
- Slow lines (thousands of ms) = the database is slow — usually memory.

When something crashes (500) there is also a line like this — the `err` part is the real reason:

```
{"t":"2026-10-02T03:41:56Z","level":"error","msg":"Unhandled error",
 "path":"/api/pickups","err":"Arithmetic overflow error converting bigint to data type
numeric."}
```

Tip

Copy this line when you ask for help — it tells exactly which address failed and why.

7. Module map — which screen uses which files

One row per module. **Screen address** = what you type after `localhost:5173`. **Screen file** = the page to change for looks/labels. **API** = the address seen in F12 → Network. **Server files** = rules and SQL (`*.routes.js` checks the input, `*.service.js` does the work). **Main tables** = where the data is.

Module	Screen address	Screen file (client/src/…)	API	Server files (server/src/modules/…)	Main tables

Login and Dashboard	/login (Employee / Franchisee / Client)
	/ (Dashboard)
		`pages/LoginPage.jsx`
		`pages/DashboardPage.jsx`
			/api/auth
			/api/menu
				`auth/auth.routes.js`
				`auth/auth.service.js`
				`menu/menu.routes.js`
				`menu/menu.service.js`
					users, employee_master, MasterMenu, SubMenu, Employee_Permission

Users, Access Rights, Change Password	/admin/users
	/admin/access-rights
	/account/change-password
		`pages/admin/UserMasterPage.jsx`
		`pages/admin/AccessRightsPage.jsx`
		`pages/account/ChangePasswordPage.jsx`
			/api/users	`users/users.routes.js`
				`users/users.service.js`
					users, Employee_Permission, SubMenu

General Masters (7)	/masters/state … /masters/division (one common master screen)	`pages/masters/MasterPage.jsx`	/api/masters	`masters/definitions.js`
				`masters/masters.routes.js`
				`masters/masters.service.js`
					state_master, zone_master, city_master, commodity_master, item_master

Branch Master	/masters/branches	`pages/masters/BranchMasterPage.jsx`	/api/branches	`branches/branches.routes.js` `branches/branches.service.js`	branch_master, New_Location_master, tblAccountLedger
Vendor Master	/masters/vendors	`pages/masters/VendorMasterPage.jsx`	/api/vendors	`vendors/vendors.routes.js` `vendors/vendors.service.js`	vendors_master, tblAccountLedger
Vehicle Master	/masters/vehicles	`pages/masters/VehicleMasterPage.jsx`	/api/vehicles	`vehicles/vehicles.routes.js` `vehicles/vehicles.service.js`	Vechile_Master, freight_master, route, Vehicle_master, tblAccountLedger
Client Master	/masters/clients	`pages/masters/ClientMasterPage.jsx`	/api/clients	`clients/clients.routes.js` `clients/clients.service.js`	client_Master, nature_of_business_master, tblAccountLedger
Employee Master	/masters/employees	`pages/masters/EmployeeMasterPage.jsx`	/api/employees	`employees/employees.routes.js` `employees/employees.service.js`	employee_master, emp_type_master, tblAccountLedger
Client Contract	/rates/contracts	`pages/rates/ContractPage.jsx`	/api/contracts	`contracts/contracts.routes.js` `contracts/contracts.service.js`	client_contract_city / _location / _State / _zone, ClientRate_With_Slab, ClientRate_Category_Slab, Min_chargeRate_slab
ODA Charge	/rates/oda; "ODA location" field on the docket	`pages/rates/OdaChargePage.jsx`	/api/oda-charges	`oda/oda.routes.js` `oda/oda.service.js`	ODA_Charge, usp_Get_OdaRate
Vendor Rate Masters (road / air / train)	/rates/vendor-road /rates/vendor-air /rates/vendor-train	`pages/rates/VendorRatesPage.jsx`	/api/vendor-rates	`vendorrates/vendorrates.routes.js` `vendorrates/vendorrates.service.js`	vendor_road_rate_master, Vendor_Air_RateMaster, Vendor_Air_Slab_Rate, Vendor_Train_Rate, vendor_vehicle_master
Franchisee Rates (commission, pickup / delivery)	/rates/franchisee-commission /rates/franchisee-pickup-delivery	`pages/rates/FranchiseeRatesPage.jsx`	/api/franchisee-rates	`franchiseerates/franchiseerates.routes.js` `franchiseerates/franchiseerates.service.js`	Franchisee_Commission_Master, Franchisee_Commission_Master_Detail, FranchiseeCommissionMaster, freight_master

Module	Screen address	Screen file (client/src/…)	API	Server files (server/src/modules/…)	Main tables
PRS / DRS Rate Master	/rates/prs-drs	pages/rates/PrsDrsRatePage.jsx	/api/prs-drs-rates	prsdrs/prsdrs.routes.js prsdrs/prsdrs.service.js	PRS_DRS_RATE_MASTER, Sp_Insert_Prs_and_Drs_Rate, vendors_master, freight_master
Settings: bindings	/settings/branch-destinations /settings/franchisee-branches /settings/franchisee-clients	pages/settings/BranchDestinationsPage.jsx pages/settings/FranchiseeBranchPage.jsx pages/settings/FranchiseeClientPage.jsx	/api/bindings	bindings/bindings.routes.js bindings/bindings.service.js	Branch_bind_For_city, Bind_Franch_Branch, Bind_Franchisee_To_Client, Franchise_Master, MasterFranchise
Settings: docket FY and effective dates	/settings/docket-fy /settings/effective-dates	pages/settings/DocketFyPage.jsx pages/settings/EffectiveDatePage.jsx	/api/docket-fy /api/effective-dates	docketfy/docketfy.routes.js docketfy/docketfy.service.js effdate/effdate.routes.js effdate/effdate.service.js	grn_master, grn_docket_charge, mainfest_entry, Manifest_grn, dispatch_master
Settings: ODA configuration	/settings/oda-config	pages/settings/OdaConfigPage.jsx	/api/oda-config	odaconfig/odaconfig.routes.js odaconfig/odaconfig.service.js	ODA_Distance_Master, ODA_Weight_Master, ODA_Configuration_Master
Settings: shippers (docket stock)	/settings/shipper-stock /settings/gatepass-stock /settings/shipper-issue /settings/shipper-issue-franchisee	pages/settings/ShipperStockPage.jsx pages/settings/ShipperIssuePage.jsx	/api/shippers	shippers/shippers.routes.js shippers/shippers.service.js	shipper_master, GatePasshipper_master, shiper_issue_to_branch, Shipper_issue_frombranch_tobranch
Settings: approvals	/settings/special-booking-approval /settings/freight-margin-approval	pages/settings/ApprovalPages.jsx	/api/approvals	approvals/approvals.routes.js approvals/approvals.service.js	grn_master_splapl, grn_docket_charge (Discount), Freight_THC_Margin_Approval_Details, Freight_Margin_Setting, dispatch_master
Settings: vendor vehicles, MIS mail	/settings/vendor-vehicles /settings/auto-mis	pages/settings/VendorVehiclePage.jsx pages/settings/AutoMisPage.jsx	/api/vendor-vehicles /api/auto-mis	vendorvehicles/vendorvehicles.routes.js vendorvehicles/vendorvehicles.service.js automis/automis.routes.js automis/automis.service.js	vendor_vehicle_master, Train_Master, VehicleName_Master, AutoMis_setting, Client_email
Docket Booking	/operation/docket/new /modify /cancel	pages/docket/DocketBookingPage.jsx	/api/dockets	docket/docket.routes.js docket/docket.schema.js docket/docket.service.js docket/tracking.service.js	grn_master, grn_docket_charge, docket_invoice, docket_dimension, docket_insurance
Pickup (PRS)	/operation/pickup/new /list	pages/pickup/PickupEntryPage.jsx	/api/pickups	pickup/pickup.routes.js pickup/pickup.service.js	Pick_up_entry, Pick_Up_grn
Manifest	/operation/manifest/new /list	pages/manifest/ManifestEntryPage.jsx	/api/manifests	manifest/manifest.routes.js manifest/manifest.service.js	mainfest_entry, Manifest_grn
Loading Sheet	/operation/loading-sheet/new /list /docket	pages/loadingsheet/LoadingSheetPage.jsx	/api/loading-sheets	loadingsheet/loadingsheet.routes.js loadingsheet/loadingsheet.service.js	Loading_Sheet, Loading_Sheet_Details, grn_master, grn_docket_charge

Module	Screen address	Screen file (client/src/…)	API	Server files (server/src/modules/…)	Main tables
Challan (vehicle dispatch)	/operation/challan/new /list	pages/challan/ChallanEntryPage.jsx	/api/challans	challan/challan.routes.js challan/challan.service.js	dispatch_master, tblAccountTrasaction
Arrival	/operation/arrival/new /list	pages/arrival/ArrivalEntryPage.jsx	/api/arrivals	arrival/arrival.routes.js arrival/arrival.service.js	Arrival_entry, Arrival_Packg
DRS (delivery run sheet)	/operation/drs/new /list	pages/drs/DrsEntryPage.jsx	/api/drs	drs/drs.routes.js drs/drs.service.js	drs_entry_details, drs_entry_details_grn
Delivery Status	/operation/delivery/new /list	pages/delivery/DeliveryStatusPage.jsx	/api/delivery	delivery/delivery.routes.js delivery/delivery.service.js	delivery_status_entry_grn, delivery_status_entry_details
POD (dispatch, head office, receive)	/operation/pod/* /pod-courier/* /pod-receive/*	pages/pod/PodDispatchPage.jsx	/api/pod /api/pod-courier	pod/pod.routes.js pod/pod.service.js podho/podho.routes.js podho/podho.service.js	pod_dispatch, pod_dispatch_image, POD_Dispatch_H / _D, pod_received
Tracking and Reports	/tracking /reports/docket-register	pages/TrackingPage.jsx pages/reports/ReportPage.jsx	/api/reports	reports/reports.routes.js reports/reports.service.js	docket_tracking, grn_master
Client Bill (GST)	/invoice/client-bill/new /invoice/client-bill/list /print/client-bill/:billNo	pages/invoice/ClientBillPage.jsx pages/invoice/ClientBillListPage.jsx pages/invoice/ClientBillPrintPage.jsx	/api/client-bills	clientbill/billstages.service.js clientbill/clientbill.routes.js clientbill/clientbill.service.js	client_bill, client_bill_charge_details, client_bill_cancel, Sac_Master, grn_master
Bill Receipt	/invoice/bill-receive	pages/invoice/BillReceivePage.jsx	/api/bill-receipts	billreceive/billreceive.routes.js billreceive/billreceive.service.js	Bill_Receive, Bill_Receive_Details, client_bill (Payment_Status), tblAccountTrasaction, tblAccountLedger
Miscellaneous and Warehouse Bills	/invoice/misc-bill /invoice/warehouse-bill /print/other-bill/:kind/:billNo	pages/invoice/OtherBillPage.jsx	/api/other-bills	otherbill/otherbill.routes.js otherbill/otherbill.service.js	miscellaneous_bill, warehouse_bill, Misc_Tax_Slab, Sac_Master, client_Master
Octroi Bills	/invoice/octroi-bill /print/octroi-bill/:billNo	pages/invoice/OctroiBillPage.jsx	/api/octroi-bills	octroibill/octroibill.routes.js octroibill/octroibill.service.js	octroi_bill, octroi_bill_charge_details, grn_master (Octroi_Status, Octroi_Bill_No)
Analytics Dashboard (86 analyses, click-through)	/ (Overview	pages/DashboardPage.jsx	/api/dashboard	dashboard/dashboard.routes.js dashboard/dashboard.service.js	grn_master, grn_docket_charge, delivery_status_entry_details, mainfest_entry, dispatch_master
Franchisee Commission Invoices	/invoice/franchisee-invoice /invoice/franchisee-invoice/cancel /print/franchisee-invoice/:invoiceNo	pages/invoice/FranchiseeInvoicePage.jsx	/api/franchisee-invoices	franchiseeinvoice/franchiseeinvoice.routes.js franchiseeinvoice/franchiseeinvoice.service.js	Franchisee_Invoice, Franchisee_Invoice_Detail, grn_master (Franchisee_Invoice_Status), Franchisee_Commission_Master (+ _Detail), Franchise_Master

Module	Screen address	Screen file (client/src/…)	API	Server files (server/src/modules/…)	Main tables

Vendor Invoices	/invoice/vendor-invoice
	/invoice/vendor-verification
	/print/vendor-invoice/:invoiceNo
	/print/vendor-bill/:billNo
		`pages/invoice/VendorInvoicePage.jsx`
		`pages/invoice/VendorVerifyPage.jsx`
			/api/vendor-invoices	`vendorinvoice/vendorinvoice.routes.js`
				`vendorinvoice/vendorinvoice.service.js`
				`vendorinvoice/vendorverify.service.js`
					Other_Vendor_Master, ProductMaster, OtherVendorInvoice (+ _Details, CostProfit), Vendor_Invoice_Verify (+ _Details)

8. What to change for what

I want to change …	Open this file	Notes
A label, a box, a column, a button on a page	`client/src/pages/<area>/<Page>.jsx` (module map, ch. 7)	Only looks — no rule changes.
Which boxes are required / the message for a box	`server/src/modules/<m>/<m>.routes.js` (zod) and the page's Form `rules`	Change both so the screen and the API agree.
A calculation, a business rule, what gets saved	`server/src/modules/<m>/<m>.service.js`	Most real logic lives here. Search for the message text.
A new column / table in the database	A **new** file `server/db/dev/NN-name.sql`, then the service	Never edit old dev SQL that is already applied; add a new numbered file and apply it.
A menu entry	`server/db/dev/99-seed.sql` (SubMenu, Employee_Permission), `client/src/routes/legacyRoutes.js`, `client/src/App.jsx`	The menu comes from the database; the link maps to the new page.
A new page	`client/src/App.jsx` (Route) + new page file + `legacyRoutes.js`	Copy a similar page as a start.
A dashboard chart / number	`server/src/modules/dashboard/dashboard.service.js` + `client/src/pages/DashboardPage.jsx`	Charts: `client/src/components/charts/Charts.jsx`.
Tracking problems (what counts as a "problem")	`server/src/modules/docket/tracking.service.js` (findIssues)	Each check says which field and who entered it.
Colours / look of all pages	`client/src/theme/themes.js`	20 ready-made looks; chosen with the palette button.
Database name, ports, secret key	`server/.env`	Never share this file. Example: `server/.env.example`.

After every change — 5 steps

1. Save the file.

2. Server file changed? Restart the API (stop it, start it again). Screen file changed? The browser updates by itself (else Ctrl + F5).

3. Try the screen again.

4. Run the automatic tests: `npm --prefix D:\rr\ramas-modern\server test` — all must pass.

5. Note what you changed (docs/MIGRATION.md) so the next person knows.

9. Checking the data in the database

Open PowerShell and run one of these (change the number in quotes). They only **read** — they change nothing.

A docket

```
sqlcmd -S .\SQLEXPRESS -E -d ramas_modern_dev -Q "select * from grn_master where docket_no =
'100000001'"
sqlcmd -S .\SQLEXPRESS -E -d ramas_modern_dev -Q "select * from grn_docket_charge where
Docket_No = '100000001'"
```

Where a docket went (manifest, challan, arrival, delivery)

```
sqlcmd -S .\SQLEXPRESS -E -d ramas_modern_dev -Q "select * from Manifest_grn where docket_no =
'100000001'"
sqlcmd -S .\SQLEXPRESS -E -d ramas_modern_dev -Q "select * from delivery_status_entry_details
where docket_no = '100000001'"
```

Easier: the Tracking page

Type the docket number in **Track docket no.** at the top. The **Entries grid** shows every entry with its values, who entered it, and the wrong ones in red.

A bill and its money

```
sqlcmd -S .\SQLEXPRESS -E -d ramas_modern_dev -Q "select * from client_bill where bill_no =
'BLBOM2627000001'"
sqlcmd -S .\SQLEXPRESS -E -d ramas_modern_dev -Q "select * from Bill_Receive_Details where
Bill_no = 'BLBOM2627000001'"
sqlcmd -S .\SQLEXPRESS -E -d ramas_modern_dev -Q "select * from tblAccountTrasaction where
VoucherNo = 'BLBOM2627000001'"
```

A user

```
sqlcmd -S .\SQLEXPRESS -E -d ramas_modern_dev -Q "select user_id, user_type, user_branch,
DivCode from users"
```

Do not

Do not delete or update rows by hand unless you are sure — use the screens (cancel / modify). Never drop the database `ramas_modern_dev`.

10. Common problems and their fixes

Problem	Why	Fix
Everything is very slow; "Query timeout"	PC low on RAM (SQL Server starved)	Close programs (Task Manager → Memory). Retry.
API keeps saying "Restarting …" and never answers	`npm run dev` (watch mode) restarts in a loop when RAM is low	Use `npm --prefix D:\rr\ramas-modern\server start` instead; restart it by hand after code changes.
"Port 4000 / 5173 already in use"	An old copy is still running	Close the old window, or find it: `Get-NetTCPConnection -LocalPort 4000` → stop that process.
PowerShell cannot open http://localhost:4000	Name "localhost" not resolving in PowerShell	Use http://127.0.0.1:4000 instead.
"Invalid user id or password" but it is correct	Wrong division / login type, or user missing in that division	Pick the right login type + division; check the `users` table.
Logged in but every list is empty	Logged in with a different financial year, or another branch selected	Log in with the current year; choose the branch at the top.
"Ledgers missing" when finalizing / receiving	An account ledger is not created for this year/division	Create it in `tblAccountLedger` (same name, year, division) — see the message for the exact name.
Too many login attempts (429)	Login protection (20 / 15 min)	Wait 15 minutes.
Changes not visible on the screen	Old page cached, or API not restarted	Ctrl + F5 in the browser; restart the API.
Tests fail with "Query timeout" or "insufficient memory"	Low RAM	Free RAM, run the tests again.
Dashboard has no / too much demo data	Demo rows are tagged DMO…	Remove: `sqlcmd -S .\SQLEXPRESS -E -d ramas_modern_dev -i server\db\dev\demo\demo-remove.sql`
Test link (trycloudflare) does not open	PC slept, internet dropped, or tunnel stopped	Run `D:\rr\tunnel\start-tunnel.ps1`; it prints the NEW link. Keep the PC awake.
Built site shows a blank page	`CLIENT_DIST` not set or the build is old	Run `scripts\make-azure-package.ps1` again; check `CLIENT_DIST`.

11. How to ask Claude (or anyone) for help

Give these 6 things — then the problem can usually be found in minutes:

#	What to give	Example
1	Which page (the address in the browser)	`localhost:5173/invoice/client-bill/new`
2	What you did, step by step	Chose client ACME, dates 1–30 Sept, clicked Search, ticked 3 dockets, clicked Generate
3	What you expected	A bill for 3 dockets
4	What happened	Red message "Docket TB003 has a GST mismatch"
5	F12 → Network → the red `/api/…` row: number + Response text, or the API log line	`POST /api/client-bills 422 {"error":"…"}`
6	The number of the docket / bill / challan, and a screenshot	Docket TB003

Message you can copy

"Page: <address>. I did: <steps>. Expected: <…>. Got: <message / number>. F12 Response: <text>. Docket/Bill: <no>. Screenshot attached."

12. Command cheat sheet

Open **PowerShell** and paste. `D:\rr\ramas-modern` is the project folder.

To …	Command
Start the API (normal)	`npm --prefix D:\rr\ramas-modern\server start`
Start the API (auto-restart on save; needs free RAM)	`npm --prefix D:\rr\ramas-modern\server run dev`
Start the screens	`npm --prefix D:\rr\ramas-modern\client run dev`
Check the API	open http://localhost:4000/api/health → `{"ok":true}`
Run all automatic tests	`npm --prefix D:\rr\ramas-modern\server test`
Apply a new dev SQL file	`sqlcmd -S .\SQLEXPRESS -E -l 60 -d ramas_modern_dev -i D:\rr\ramas-modern\server\db\dev\39-name.sql`
Load / remove demo data	`sqlcmd -S .\SQLEXPRESS -E -d ramas_modern_dev -i …\server\db\dev\demo\demo-data.sql` (or `demo-remove.sql`)
Build the app for hosting	`powershell -ExecutionPolicy Bypass -File D:\rr\ramas-modern\scripts\make-azure-package.ps1`
Start / stop the public test link	`powershell -ExecutionPolicy Bypass -File D:\rr\tunnel\start-tunnel.ps1` (`stop-tunnel.ps1`)
Update the project document (flow.docx)	`node docs\tools\make-flow-doc.cjs` (needs the `docx` package)
Make this guide again	`node docs\tools\make-debug-guide.cjs`, then save as PDF in Word
Is SQL Server running?	`Get-Service MSSQL$SQLEXPRESS`
How much RAM is free?	`(Get-CimInstance Win32_OperatingSystem).FreePhysicalMemory / 1MB` (GB)

13. Where everything lives (folders)

```
D:\rr\ramas-modern
├─ client\                     the SCREENS (React)
│  └─ src\
│     ├─ App.jsx               list of all page addresses (Routes)
│     ├─ pages\                one file per page  (invoice\, operation\, rates\, settings\ …)
│     ├─ components\charts\    dashboard charts (ECharts) and the click-through drawer
│     ├─ routes\legacyRoutes.js  old .aspx page → new address
│     ├─ theme\themes.js       the 20 looks
│     └─ api\client.js         how the screens call the API
├─ server\                     the API (Node.js)
│  ├─ .env                     settings + database connection (private)
│  ├─ src\app.js               all API addresses are connected here
│  ├─ src\modules\<name>\      one folder per module:
│  │                           <name>.routes.js = checks, <name>.service.js = work + SQL
│  ├─ src\middleware\          login check, error handling
│  ├─ db\dev\NN-*.sql          tables + procedures, applied in number order
│  │                           (99-seed.sql = sample masters, users, menu)
│  ├─ db\dev\demo\             demo data (DMO…) and its remover
│  └─ test\api.test.js         the automatic tests
├─ docs\                       MIGRATION.md (what was built), flow.docx,
│                              SCHEMA-VERIFICATION.md, this guide
└─ scripts\make-azure-package.ps1   builds the app for hosting

D:\rr\tunnel\     public test link (start/stop scripts, TEST-LOGINS.txt — private)
C:\projects\ramas-modern\original\   the OLD ASP.NET application (read only, for reference)
```

14. Words used in this guide

Word	Meaning
API	The middle part of the app (server). Screens ask it to read or save data.
Endpoint / address	One "door" of the API, e.g. `/api/dockets`.
Module	One area of the app with its own folder, e.g. client bill, tracking, dashboard.
Route (screen)	A page address like `/invoice/octroi-bill`, listed in `App.jsx`.
routes.js	API file that lists the module's addresses and checks the input.
service.js	API file that does the work: rules, calculations, SQL.
Database / table	Where data is stored; a table is like an Excel sheet (rows and columns).
Stored procedure	A saved SQL program in the database (from the old app); in `server/db/dev/*.sql`.
Transaction	Several saves done as one: either all succeed or none.
F12 / DevTools	The browser tool that shows errors (Console) and requests (Network).
Status number	200/201 = OK; 400–499 = the request was refused; 500 = the server failed.
Dev database	`ramas_modern_dev` — the practice database on this PC (not the real one).
Test database	`ramas_test` — used by the public test link; has its own passwords.
Demo data	Made-up records tagged `DMO…` so the dashboard has something to show.

— End of guide —